

Artificial Intelligence

Section 7

By: Mohamed Fathy

Section Content

- Greedy Best-first Search
- A^* Search
- Heuristic Functions

Search Algorithm

```
graph TD; SA([Search Algorithm]) --> UB[Uniformed/Blind]; SA --> IS[Informed Search]; UB --> UB1[ ]; UB --> UB2[ ]; UB --> UB3[ ]; UB --> UB4[ ]; UB --> UB5[ ]; UB --> UB6[ ]; IS --> BFS[Best First Search]; IS --> AS[A*search];
```

Uniformed/Blind

Informed Search

Best First Search

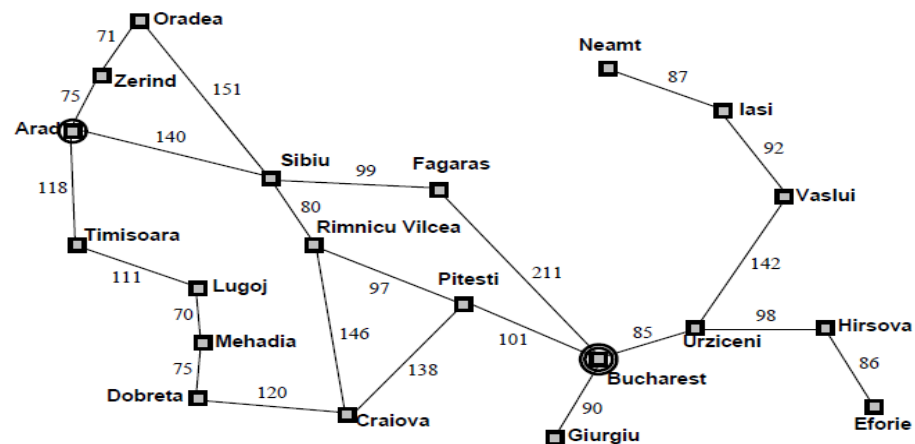
A*search

- **Greedy best-first search** is a form of best-first search that expands first the node with the lowest $h(n)$ “**heuristic**” value—the node that appears to be closest to the goal—on the grounds that this is likely to lead to a solution quickly. So, the evaluation function $f(n) = h(n)$.
- Let us see how this works for route-finding problems in Romania; we use the straight-line distance heuristic, which we will call h_{SLD} . If the goal is Bucharest, we need to know the straight-line distances to Bucharest.
- Notice that the values of h_{SLD} cannot be computed from the problem description itself. Moreover, it takes a certain amount of world knowledge to know that h_{SLD} is correlated with actual road distances and is, therefore, a useful heuristic.

Figure 3.16

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.



Greedy Best-first Search

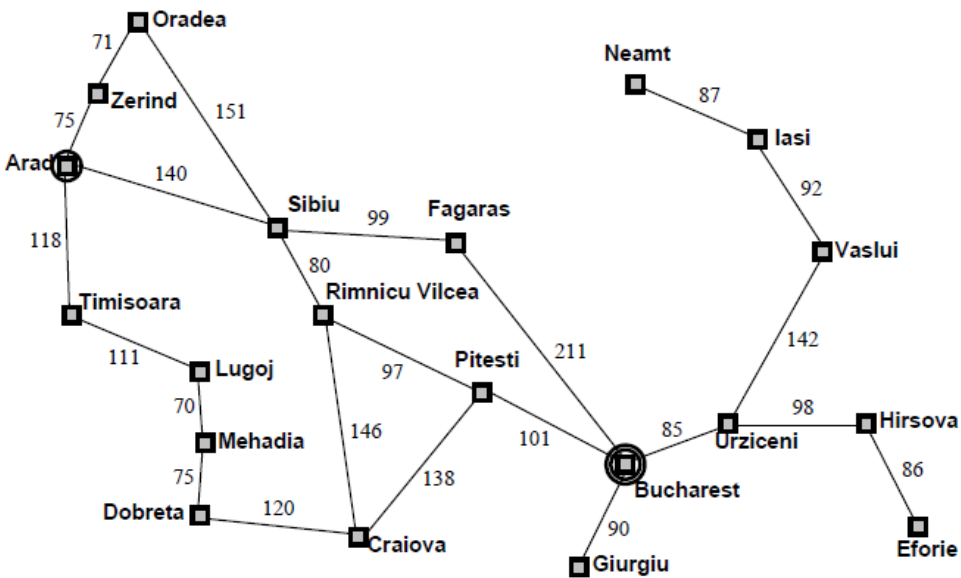
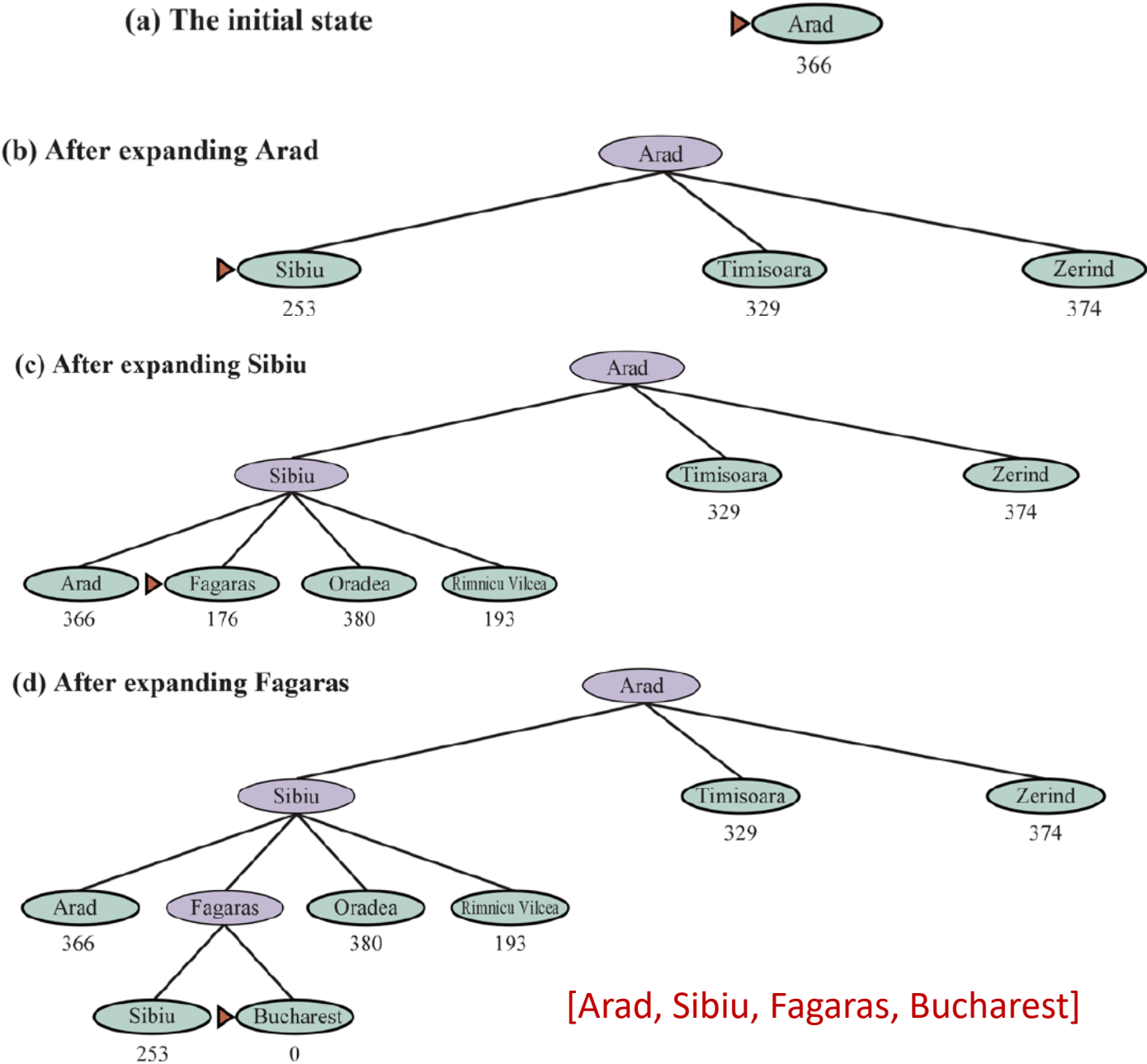


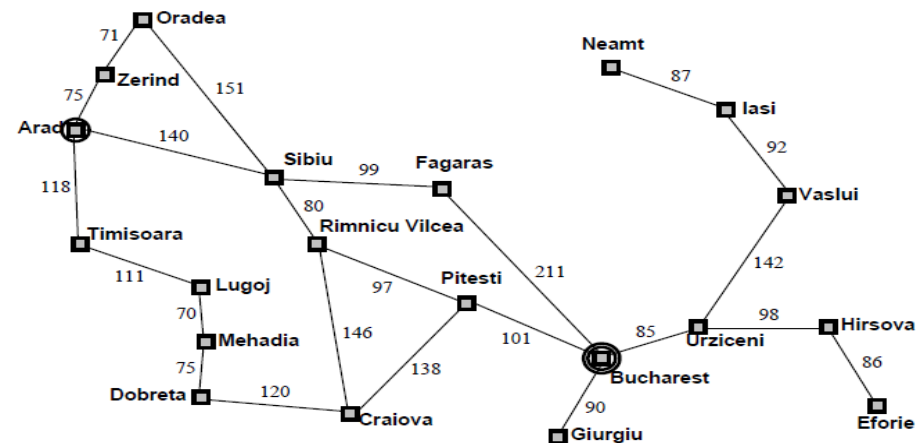
Figure 3.16

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

- For this particular problem, greedy best-first search using h_{SLD} finds a solution without ever expanding a node that is not on the solution path. **The solution it found does not have optimal cost.**
- This is why the algorithm is called “greedy”—on each iteration it tries to get as close to a goal as it can, but greediness can lead to worse results than being careful.
- Greedy best-first graph search is complete in finite state spaces, but not in infinite ones.
- The worst-case time and space complexity is $O(|V|)$. With a good heuristic function, however, the complexity can be reduced substantially, on certain problems reaching $O(bm)$

- For this particular problem, greedy best-first search using h_{SLD} finds a solution without ever expanding a node that is not on the solution path. **The solution it found does not have optimal cost.**
- This is why the algorithm is called “greedy”—on each iteration it tries to get as close to a goal as it can, but greediness can lead to worse results than being careful.
- Greedy best-first graph search is complete in finite state spaces, but not in infinite ones.
- The worst-case time and space complexity is $O(|V|)$. With a good heuristic function, however, the complexity can be reduced substantially, on certain problems reaching $O(bm)$



Greedy Best-first Search

Romania Problem Code

**Greedy Best-first
Search**

Search Algorithm

```
graph TD; SA([Search Algorithm]) --> UB[Uniformed/Blind]; SA --> IS[Informed Search]; UB --> UB1[ ]; UB --> UB2[ ]; UB --> UB3[ ]; UB --> UB4[ ]; UB --> UB5[ ]; UB --> UB6[ ]; IS --> IS1[ ]; IS --> AS[A*search]; AS --> AS2[ ]
```

Uniformed/Blind

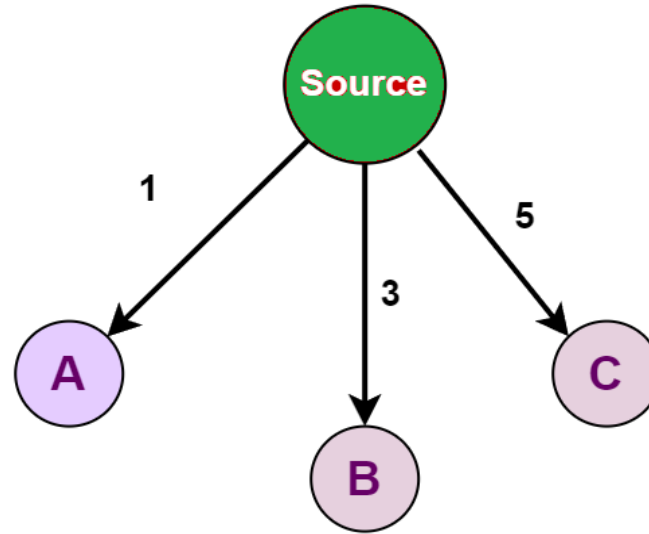
Informed Search

A*search

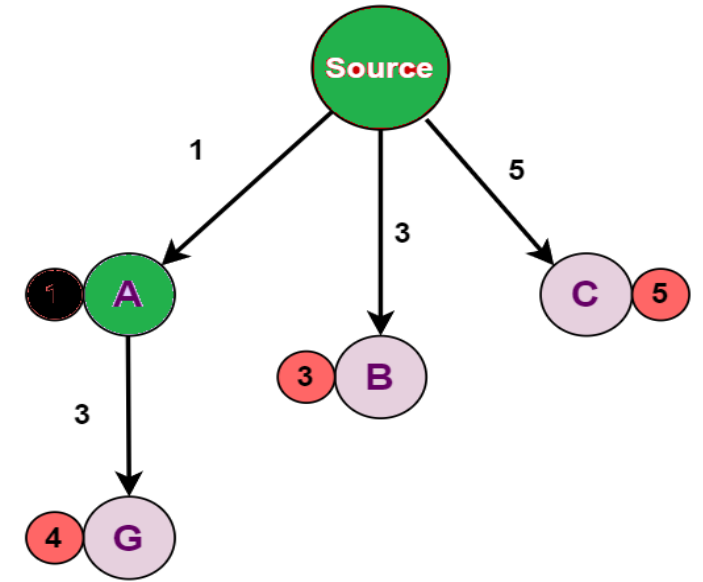
Visited Nodes: –



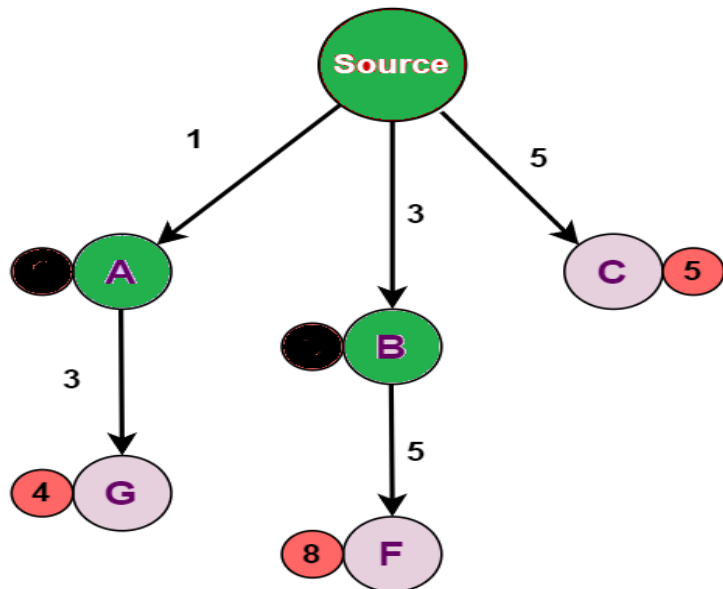
Visited Nodes: source



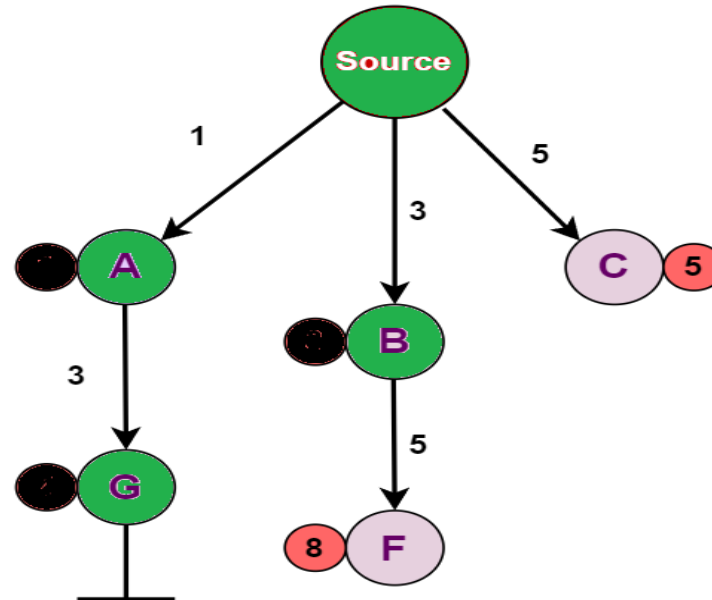
Visited Nodes: source, A



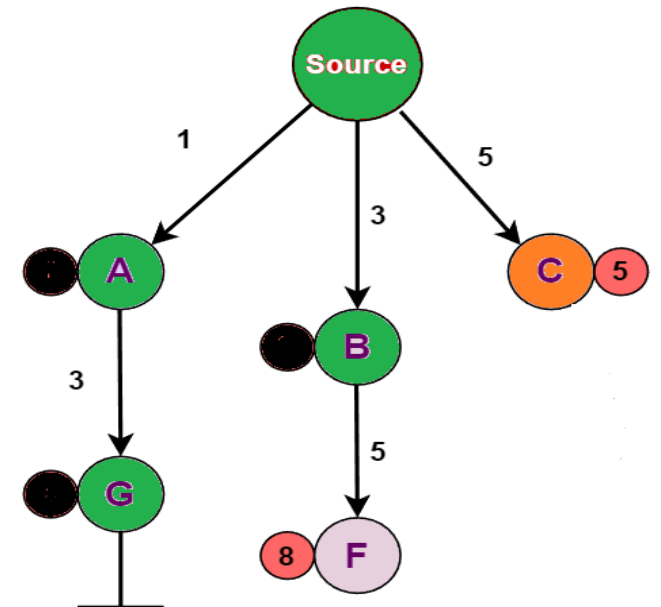
Visited Nodes: source, A, B



Visited Nodes: source, A, B, G



Visited Nodes: source, A, B, G, C



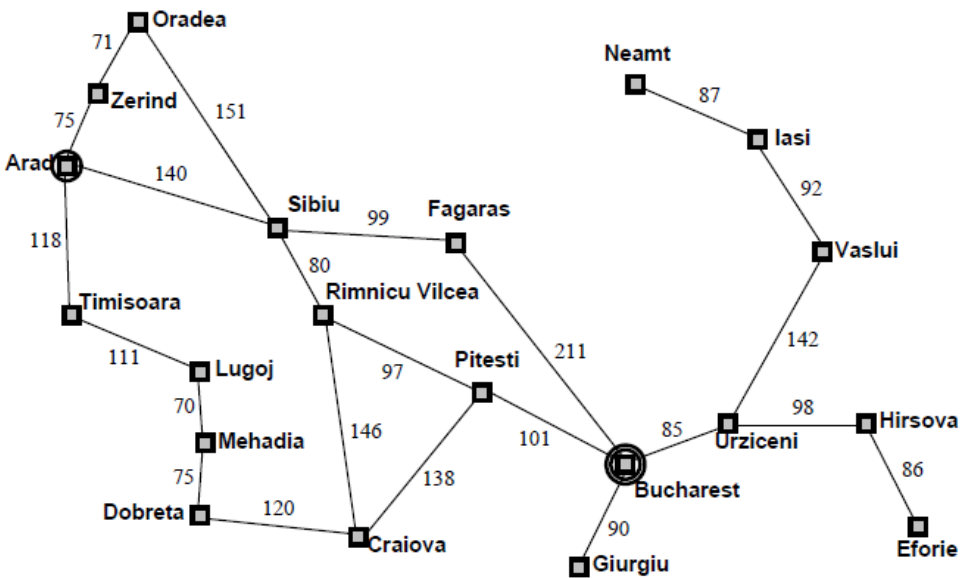
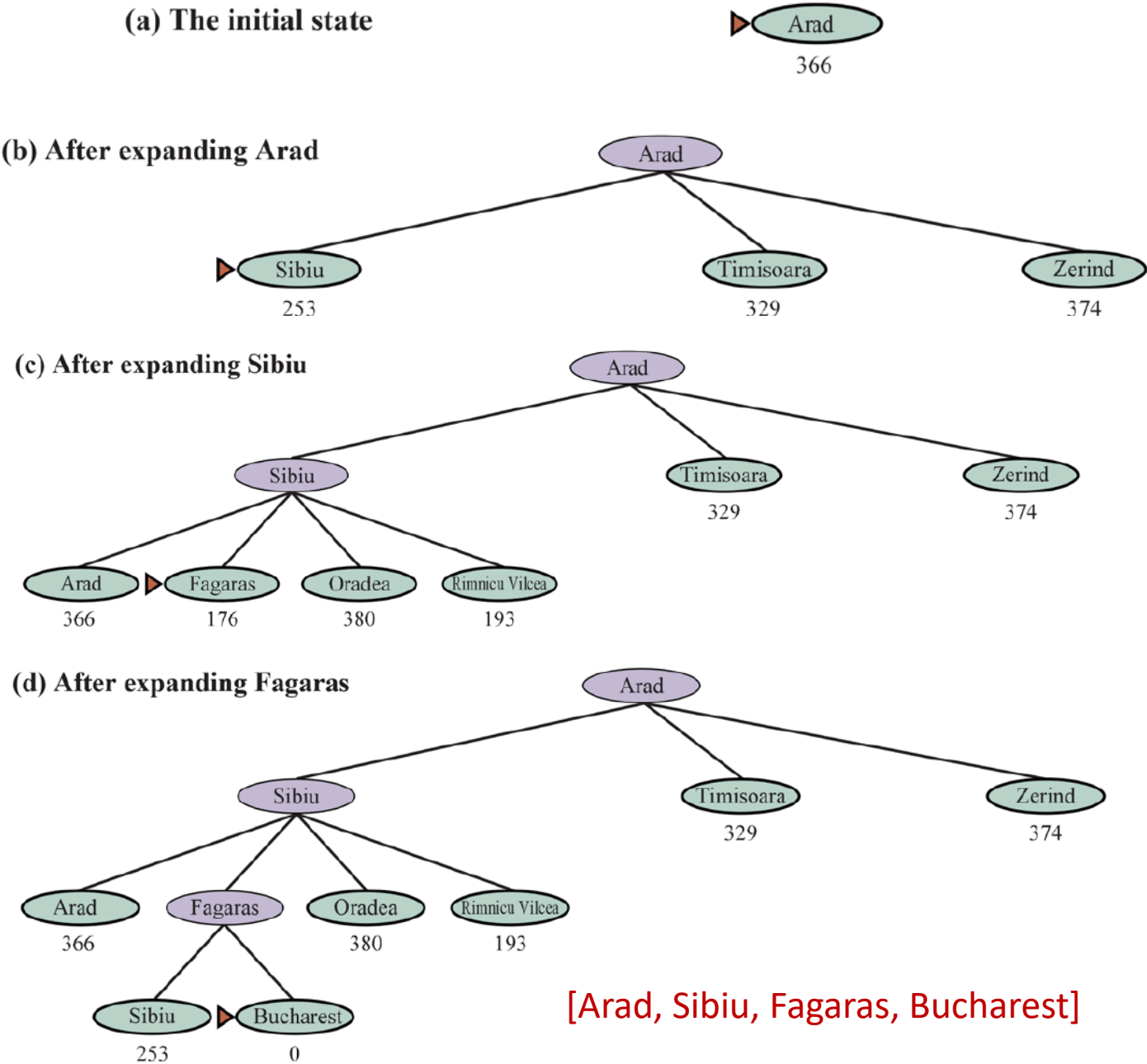


Figure 3.16

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

- The most common informed search algorithm is A* search (pronounced “A-star search”), a best-first search that uses the evaluation function

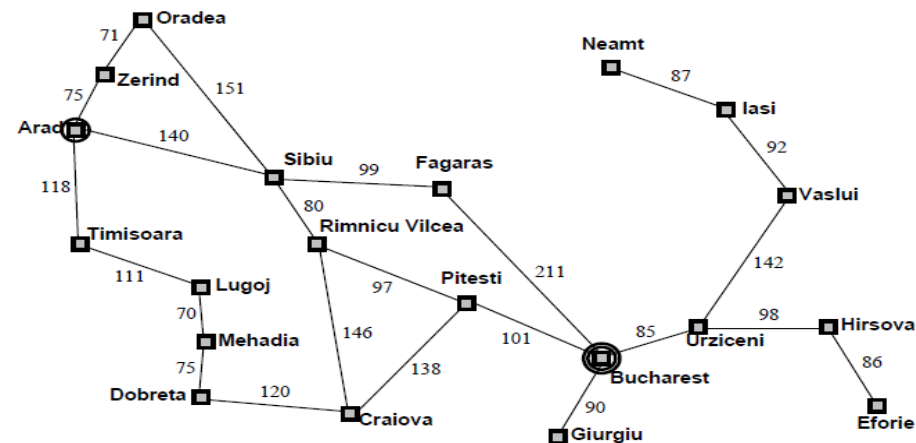
$$f(n) = g(n) + h(n)$$

where $g(n)$ is the path cost from the initial state to node n , and $h(n)$ is the estimated cost of the shortest path from n to a goal state

Figure 3.16

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

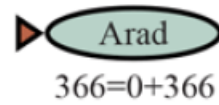
Values of h_{SLD} —straight-line distances to Bucharest.



A* Search

Heuristic
Functions

(a) The initial state



(b) After expanding Arad

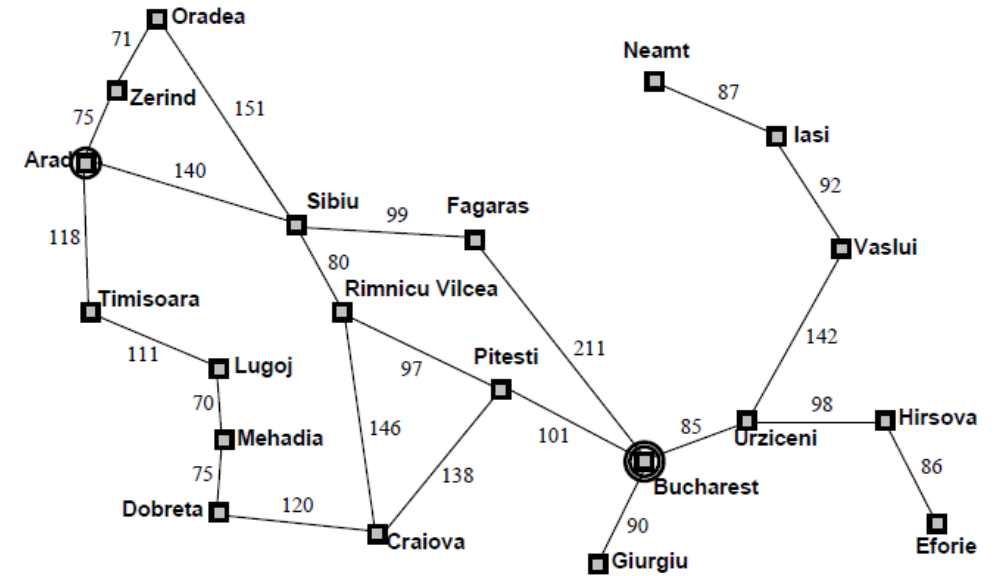
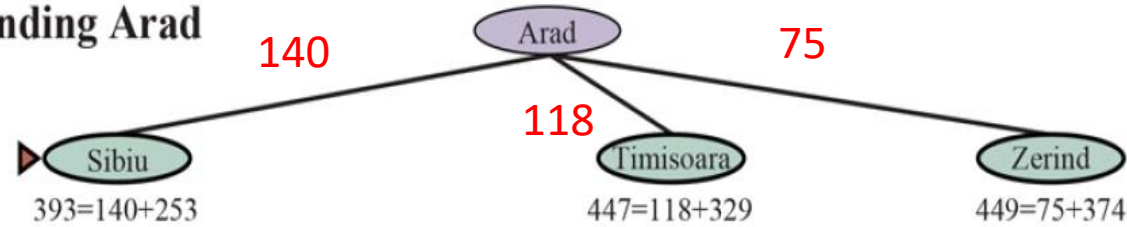
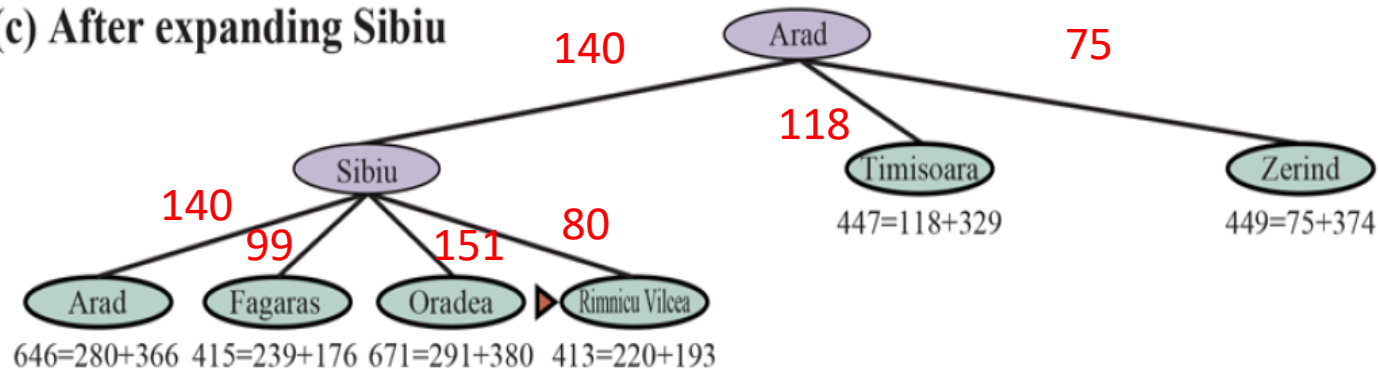


Figure 3.16

(c) After expanding Sibiu



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

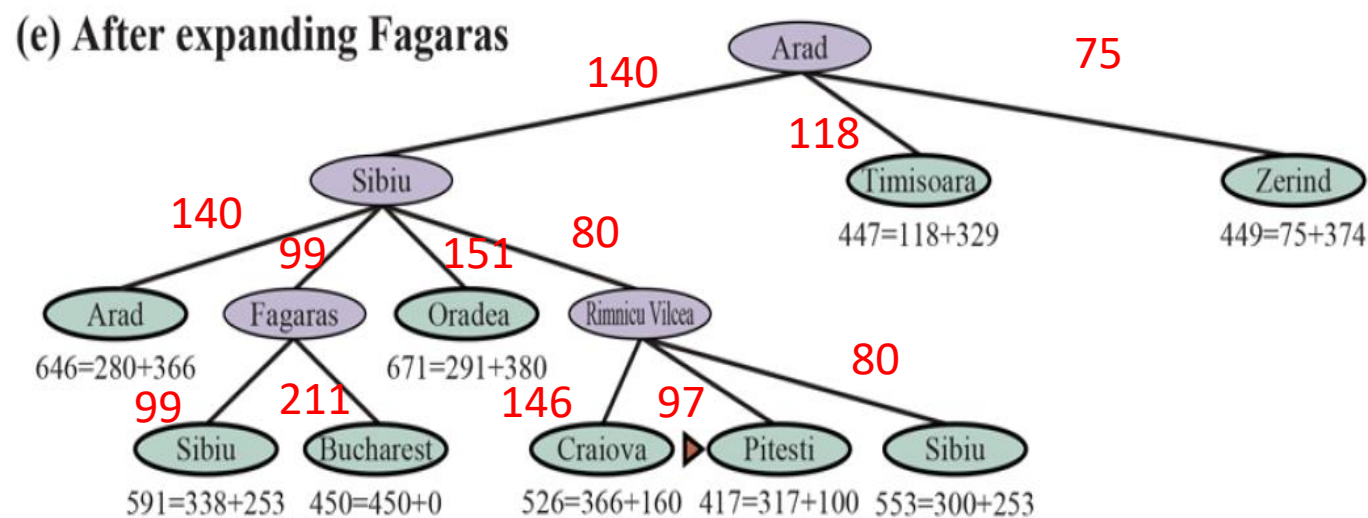
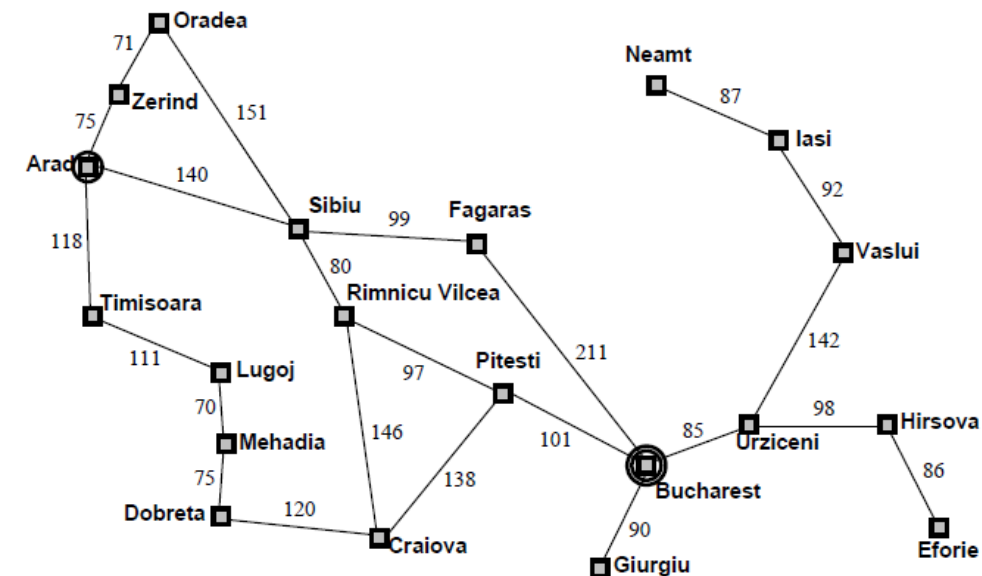
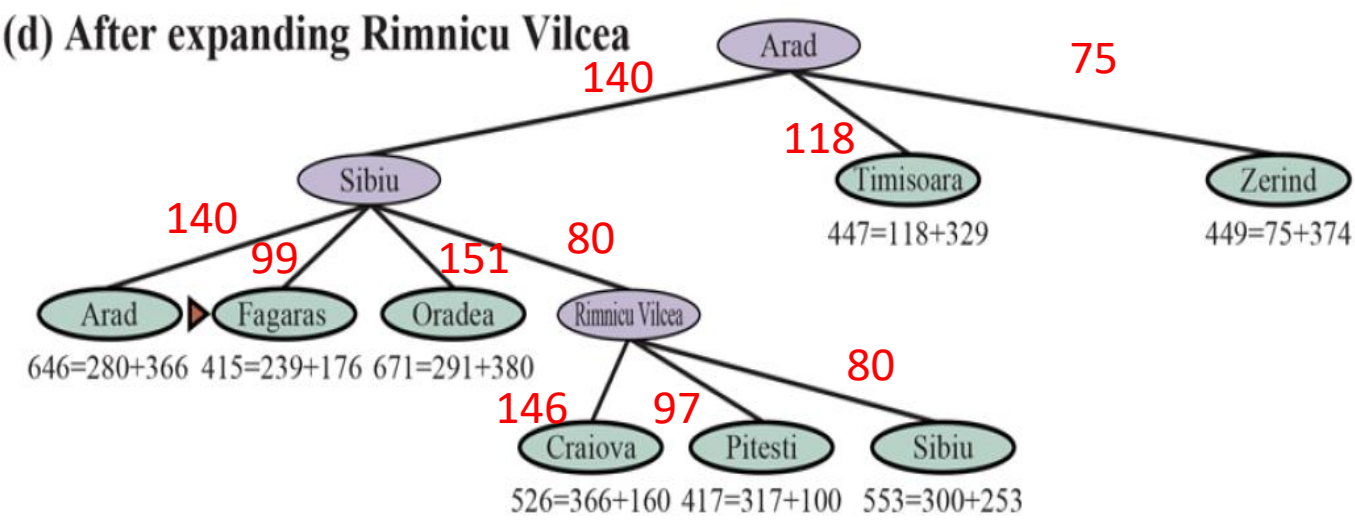
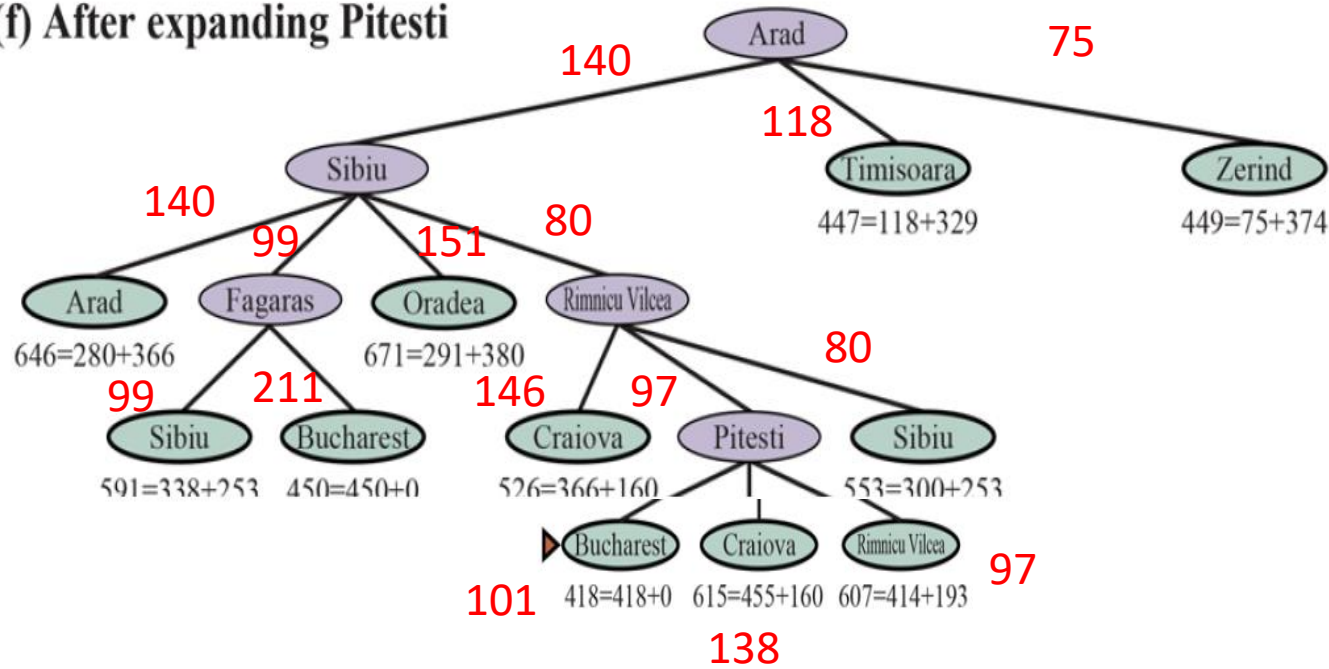


Figure 3.16

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.



[Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest]

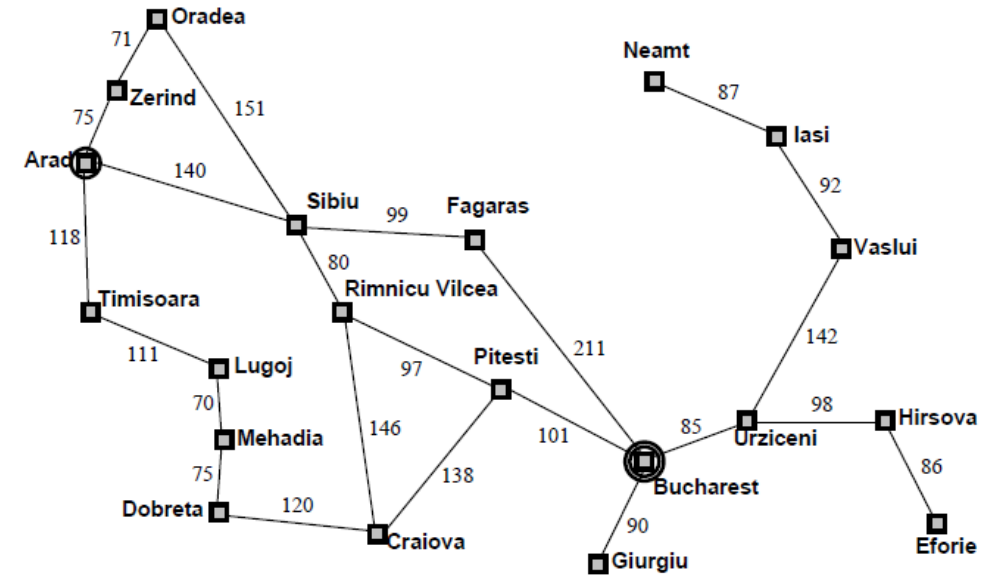


Figure 3.16

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

Romania Problem Code

A^* Search

- A key property is **admissibility**: an **admissible heuristic** is one that never overestimates the cost to reach a goal.
- A^* search is complete.
- With an admissible heuristic, A^* is cost-optimal.
- With an inadmissible heuristic, A^* may or may not be cost-optimal.

Heuristic
Functions

- In this section, we look at **how the accuracy of a heuristic affects search performance**, and also consider **how heuristics can be invented**. As our main example we'll return to the 8-puzzle.
- There are $9!/2 = 181,440$ reachable states in an 8-puzzle, so a search could easily keep them all in memory. But for the 15-puzzle, there are $16!/2$ states—over 10 trillion—so to search that space we will need the help of a good admissible heuristic function.
- There is a long history of such heuristics for the 15-puzzle; here are two commonly used candidates:

1. **h_1 = the number of misplaced tiles (blank not included):** all eight tiles are out of position, so the start state has $h_1 = 8$.
 h_1 is an admissible heuristic

2. **h_2 = the sum of the distances of the tiles from their goal positions:** Because tiles cannot move along diagonals, the distance is the sum of the horizontal and vertical distances sometimes called the **city-block distance** or **Manhattan distance**. Tiles 1 to 8 in the start state of Figure 3.25 give a Manhattan distance of $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$.
 h_2 is an admissible heuristic

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

A^* Search

Heuristic
Functions

A^* Search

8-Puzzle Codes

**Heuristic
Functions**

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

A^* Search

Heuristic
Functions

- The results suggest that h_2 is better than h_1 and both are better than no heuristic at all.